

# JAMstack: Arquitectura Moderna de Sitios Estáticos

**JAMstack (JavaScript, APIs y Markup)** es un modelo de desarrollo que **desacopla el frontend del backend**, haciendo que no se dependa de un servidor tradicional en tiempo real.

A diferencia de la arquitectura tradicional (que genera HTML dinámico en cada petición conectándose a una base de datos (como en PHP o Node tradicional)) a través de un servidor, **JAMstack pre-genera archivos HTML estáticos durante el proceso de *build* (construcción) antes de que el usuario acceda y los sirve desde un CDN**, utilizando JavaScript para llamar a APIs solo cuando se necesitan datos dinámicos, obteniendo como resultado menos carga en servidor y mayor velocidad.

## COMPONENTES CLAVE:

- **Markup:** HTML preconstruido en el momento de build, a menudo generado a partir de Markdown (ideal para blogs y documentación).
- **CMS Headless:** Sistemas (como Contentful o Strapi) que solo gestionan datos y los exponen mediante APIs, sin controlar el diseño del frontend.
- **JavaScript:** Añade interactividad en el navegador (formularios, carritos, filtros).
- **APIs:** Sustituyen al backend tradicional, permitiendo obtener datos externos de forma dinámica durante el *build* o en tiempo real en el cliente.

## FLUJO COMPLETO:

- 1. El desarrollador escribe el sitio.
- 2. Se ejecuta un build.
- 3. Se generan archivos HTML/CSS/JS.
- 4. Se suben a un CDN.
- 5. El usuario accede rápidamente.
- 6. JavaScript llama APIs si hace falta.

## **HERRAMIENTAS COMUNES**

- **Generadores (SSG)**
  - Next.js -> híbrido (SSR + SSG).
  - Astro -> muy rápido, moderno.
  - Gatsby -> basado en GraphQL.
- **CMS Headless** -> Separan contenido de presentación.
  - Contentful.
  - Strapi.
- **Hosting / Deploy** -> Usan CDN para servir contenido globalmente.
  - Netlify.
  - Vercel.
  - GitHub Pages.

## **VENTAJAS:**

Alto rendimiento (carga casi instantánea), mayor seguridad (sin base de datos expuesta), escalabilidad sencilla y costes de hosting más bajos.

## **DESVENTAJAS:**

Menos adecuado para aplicaciones con lógica compleja en tiempo real, dependencia de APIs externas y tiempos de *build* lentos en sitios muy grandes.

## **¿CUÁNDO USAR JAMSTACK?:**

Blogs, Landing pages, Portfolios, Documentación, E-commerce simple.

# Accesibilidad Web (WCAG)

Las WCAG son normas para asegurar que cualquier persona, incluyendo aquellas con discapacidades visuales, motoras, auditivas o cognitivas, pueda usar la web.

## PRINCIPIOS “POUR”:

- **Perceptible:** El usuario debe poder percibir la información (ej. texto alternativo en imágenes, subtítulos, buen contraste).
- **Operable:** El usuario debe poder interactuar con la web (ej. navegación completa con teclado, botones accesibles).
- **Comprensible:** El contenido y la interfaz deben ser fáciles de entender (ej. lenguaje claro, mensajes de error útiles).
- **Robusto:** La web debe funcionar con diferentes tecnologías, incluyendo lectores de pantalla.

## ARIA

Son atributos HTML que ayudan a las tecnologías de asistencia a entender componentes complejos (ej. modales, menús desplegables).

**Regla de oro: "Primero HTML semántico, luego ARIA si hace falta".** No se debe usar ARIA si un elemento HTML nativo (como un `<button>`) ya cumple esa función de forma accesible.

**Errores Comunes:** Botones sin etiqueta, formularios sin `<label>`, usar solo el color para comunicar información (afecta a daltónicos) y no permitir la navegación con teclado.

## **Ejemplos típicos donde SÍ usar ARIA:**

Menús desplegables.      Modales.      Tabs.

Acordeones.      Componentes hechos con JavaScript.

# SEO

El SEO busca mejorar la visibilidad de una web en buscadores como Google de forma orgánica (sin pagar anuncios).

## TIPOS DE SEO:

- **On-page:** Optimización dentro de la web (contenido relevante, palabras clave, estructura HTML).
- **Off-page:** Factores externos, principalmente la obtención de *backlinks* de calidad para ganar autoridad.
- **Técnico:** Optimización de la parte "invisible" (velocidad de carga, indexación, estructura) para que Google entienda la web.

## TÉCNICAS CLAVE:

- **Optimizar <title> y <meta name="description">.** *(Un buen título es claro, incluye palabra clave, y tiene entre 50-60 caracteres).*
- **Usar una estructura jerárquica de encabezados** (<h1> a <h6>), con un solo <h1> por página.
- **Utilizar URLs limpias** (ej. /curso-seo).
- Configurar sitemap.xml y robots.txt.
- **Implementar** datos estructurados (**Schema.org**) en JSON-LD para habilitar *Rich Snippets* (resultados enriquecidos con valoraciones o FAQs).

**CORE WEB VITALS:** Métricas de Google para medir la experiencia de usuario.

- **LCP (Largest Contentful Paint):** Mide el tiempo de carga del contenido principal (debe ser < 2.5s).
- **CLS (Cumulative Layout Shift):** Mide la estabilidad visual (cambios inesperados en el diseño).
- **INP (Interaction to Next Paint):** Mide el tiempo de respuesta al interactuar con la página.
- **Factores importantes adicionales:**
  - Mobile-first (diseño para móvil).

- HTTPS (seguridad).
  - Contenido de calidad.
  - Tiempo de permanencia del usuario.

# Diseño en Figma

Figma es una herramienta de diseño en la nube para crear interfaces, prototipos interactivos y trabajar colaborativamente en tiempo real.

## ELEMENTOS DEL DISEÑO:

- **Wireframes:** Bocetos simples (en blanco y negro) enfocados en definir la estructura y el *layout*.
- **UI Design:** Diseño visual final de la interfaz, añadiendo colores, tipografía, iconos y espaciado.
- **Componentes reutilizables:** Elementos que se repiten (botones, tarjetas, navbar) para ahorrar tiempo y mantener la consistencia.
- **Design Systems:** Conjunto de reglas (colores oficiales, tipografías) y componentes que rigen todo el diseño de un proyecto.

## PROTOTIPADO:

Permite crear flujos de navegación conectando pantallas y añadir interacciones (click, hover, scroll) para realizar *testing* de UX antes de programar.

## BUENAS PRÁCTICAS:

Mantener la consistencia visual, establecer una jerarquía visual clara (destacar lo importante) y aplicar un enfoque *mobile-first* (diseñar primero para móvil y luego escalar a escritorio).

## TESTING DE UX: Probar si el diseño funciona bien.

- ¿Es fácil usarlo?
- ¿Se entiende rápido?
- ¿El usuario se pierde?